WalletPlatform

High Level Design Document

Microservices Wallet & Payment Platform

Document Version	1.0
Status	Final
Technology Stack	.NET 8 | Angular 17 | SQL Server | RabbitMQ
Architecture	Microservices with Event-Driven Communication
Classification	Training Project

1. Executive Summary

WalletPlatform is a production-grade digital wallet and payment system modelled after platforms such as Paytm and PhonePe. It enables users to create wallets, fund them, transfer money peer-to-peer, earn loyalty rewards, and redeem points for catalog items — all through a secure, scalable microservices architecture.

The platform is built on .NET 8 using ASP.NET Core Web APIs, with Angular 17 as the frontend. It uses SQL Server for persistence, RabbitMQ for asynchronous event-driven communication, and Ocelot as the API gateway. The system is designed to be independently deployable, horizontally scalable, and resilient to partial failures.

Key Business Capabilities

- Secure user registration, authentication, and KYC (Know Your Customer) verification
- Digital wallet creation, funding, peer-to-peer transfers, and withdrawal
- Immutable double-entry ledger with full audit trail
- Idempotent payment processing with Saga-based compensation
- Configurable loyalty rewards engine with tier-based point multipliers
- Catalog-based points redemption with voucher code generation
- Multi-channel notification delivery (Email and SMS)
- Admin capabilities: KYC review, wallet freeze/unfreeze, rule management

2. Business Requirements

2.1 Functional Requirements

Authentication & Identity

- Users must be able to register with email, phone number, and password
- System must enforce strong password policy (uppercase, number, special character)
- JWT-based authentication with 60-minute access tokens and 7-day refresh tokens
- Role-based access control: User, Admin, and Agent roles
- KYC document submission, admin review, approval, and rejection workflow
- Wallet activation must be automatic upon KYC approval

Wallet & Payments

- Every user gets one wallet created automatically on registration
- Wallet remains in Pending state until KYC is approved
- Users can fund wallets from external payment methods
- Peer-to-peer transfers must be atomic and idempotent
- Admins can freeze and unfreeze wallets with an audit reason
- All balance operations must be protected against concurrent modification

Ledger & Accounting

- Every financial transaction must produce two ledger entries (double-entry accounting)
- Ledger entries must be immutable — no updates or deletes permitted
- Balance must be derivable from ledger history at any point in time
- Duplicate payments must be rejected using client-supplied idempotency keys

Rewards & Catalog

- Points awarded automatically on qualifying transactions
- Configurable point rules per transaction type with caps and thresholds
- Four-tier loyalty system: Bronze, Silver, Gold, Platinum
- Tier determined by lifetime points (never decreases on spending)
- Catalog items redeemable for points with voucher code generation

2.2 Non-Functional Requirements

Requirement	Target	Implementation
Availability	99.9% per service	Independent deployment, no shared failure domain
Scalability	Horizontal per service	Stateless APIs, shared-nothing architecture
Security	OWASP Top 10 compliant	JWT, BCrypt, rate limiting, input validation
Auditability	Full financial trail	Immutable ledger, audit logs, Serilog structured logs
Resilience	Graceful degradation	Circuit breaker, DLQ, Saga compensation
Response Time	< 300ms for reads	Indexed queries, async event processing
Data Integrity	Zero money loss	Idempotency, atomic ledger writes, Saga rollback

3. System Context

3.1 System Actors

Actor	Description
End User	Registers, manages wallet, sends/receives payments, earns and redeems rewards
Admin	Reviews KYC, approves/rejects, freezes wallets, manages point rules and catalog
API Gateway	Single entry point for all clients. Validates JWT, applies rate limits, routes requests
RabbitMQ Broker	Message bus for all asynchronous communication between microservices
SQL Server	Persistent storage — one database per microservice
External Providers	Email provider (SendGrid), SMS provider (Twilio) — currently simulated

3.2 System Boundaries

The system boundary is defined by the API Gateway at port 5000. All client traffic — from the Angular web application, mobile apps, or third-party integrations — enters exclusively through the gateway. Individual microservices are not publicly accessible. Internal service-to-service communication either goes through synchronous REST calls (within the internal network) or asynchronous RabbitMQ events.

4. Architecture Overview

4.1 Architectural Style

The platform uses a Microservices Architecture with Event-Driven Communication. Each service is an independently deployable unit with its own codebase, database, and deployment lifecycle. Services communicate through two channels:

Communication Type	Usage
Synchronous REST	When the caller needs an immediate response — login, balance check, payment initiation
Asynchronous Events	When the operation can complete in the background — wallet creation, rewards, notifications

4.2 Layered Architecture Within Each Service

Every microservice follows a strict three-layer architecture:

Layer	Component	Responsibility
Presentation	Controllers, Middleware	HTTP handling, request validation, response formatting
Business	Services, Event Publishers	Business logic, orchestration, event publishing
Data Access	Repositories, DbContext	SQL queries, EF Core, no business logic

4.3 Microservices Inventory

Service	Port	Database	Responsibility
Auth.API	5010	auth_db	Registration, login, JWT generation, KYC workflow, audit logging
Wallet.API	5020	wallet_db	Wallet lifecycle, balance management, freeze/unfreeze
Transaction.API	5030	ledger_db	Double-entry ledger, idempotency, Saga compensation

Service	Port	Database	Responsibility
Rewards.API	5040	rewards_db	Points engine, tier management, loyalty accounts
Catalog.API	5050	catalog_db	Rewards catalog, point redemptions, voucher generation
Notification.API	5080	notification_db	Email/SMS delivery, template rendering, delivery logs
Reporting.API	5070	reporting_db	Report definitions (extensible)
ApiGateway	5000	None	JWT validation, routing, rate limiting, circuit breaking

5. Technology Stack

5.1 Technology Decisions

Technology	Choice	Justification
Backend Framework	.NET 8 ASP.NET Core	Mature, high-performance, excellent async support, LTS release
Frontend	Angular 17	Component-based, strong TypeScript support, enterprise-grade
API Gateway	Ocelot 23.x	Native .NET gateway, Polly integration, JWT validation, rate limiting
Message Broker	RabbitMQ 3.x	Reliable delivery, dead-letter queues, management UI, AMQP standard
Database	SQL Server Express	ACID compliance, EF Core 8 native support, strong tooling (SSMS)
ORM	Entity Framework Core 8	Code First migrations, LINQ queries, change tracking, retry on failure
Authentication	JWT + BCrypt	Stateless auth for microservices, BCrypt for secure password hashing
Logging	Serilog	Structured logging, multiple sinks, enrichment, queryable output
Validation	FluentValidation 11	Separation of validation from DTOs, multiple error messages, testable
Circuit Breaker	Polly (via Ocelot)	Fast failure, automatic recovery, prevents cascade failures
Resilience Pattern	Saga (Choreography)	Distributed compensation without central orchestrator

6. Data Ownership Model

6.1 Database-Per-Service

Each microservice owns its database exclusively. No service may query another service's database directly. Cross-service data access is achieved exclusively through REST API calls or RabbitMQ event subscriptions. This enforces true service independence and allows each service to evolve its schema without affecting others.

Database	Owner Service	Key Tables
auth_db	Auth.API	Users, UserProfiles, KYCRecords, RefreshTokens, AuditLogs
wallet_db	Wallet.API	Wallets, PaymentMethods, WalletFreezeLogs
ledger_db	Transaction.API	Transactions, LedgerEntries, IdempotencyKeys
rewards_db	Rewards.API	LoyaltyAccounts, RewardTiers, PointRules, PointTransactions
catalog_db	Catalog.API	CatalogItems, Redemptions
notification_db	Notification.API	NotificationLogs, NotificationTemplates
reporting_db	Reporting.API	ReportDefinitions

6.2 Cross-Service Data Strategy

Denormalized Foreign Keys

When Service B needs to reference an entity owned by Service A, it stores only the ID as a plain column (not a database foreign key). For example, wallet_db stores userId as a Guid column without a foreign key constraint to auth_db. Referential integrity is maintained at the application level through events.

Event-Carried State Transfer

Events carry all data that consumers need to avoid synchronous lookups. For example, UserRegisteredEvent carries Email, PhoneNumber, and FullName so Notification.API does not need to call Auth.API to send a welcome email.

7. Event-Driven Communication

7.1 Event Catalog

Event	Published By	Consumed By
UserRegisteredEvent	Auth.API	Wallet.API, Rewards.API, Notification.API
KYCApprovedEvent	Auth.API	Wallet.API, Notification.API
WalletFundedEvent	Wallet.API	Transaction.API, Notification.API
TransactionCompletedEvent	Transaction.API	Rewards.API, Notification.API
TransactionFailedEvent	Transaction.API	Wallet.API (compensation), Notification.API
PointsAwardedEvent	Rewards.API	Notification.API
RedemptionRequestedEvent	Catalog.API	Rewards.API

7.2 Reliability Guarantees

Guarantee	Implementation
At-Least-Once Delivery	Manual acknowledgement (autoAck: false) — messages redelivered on failure
Durability	durable: true queues + Persistent messages survive RabbitMQ restart
Dead Letter Handling	Every queue has a DLQ (x-dead-letter-exchange) — failed messages preserved
Idempotent Consumers	Wallet creation checks for existing wallet before creating (guard clause)
Ordered Processing	BasicQos(prefetchCount: 1) — one message at a time per consumer

7.3 Queue Naming Strategy

Each service that consumes the same event gets its own uniquely named queue. For example, the user.registered event is consumed from three separate queues:

- user.registered — consumed by Wallet.API
- user.registered.rewards — consumed by Rewards.API
- user.registered.notifications — consumed by Notification.API

This fan-out pattern ensures every service receives every event independently, regardless of processing speed or availability of other consumers.

8. Security Architecture

8.1 Authentication & Authorization

Mechanism	Detail
Password Hashing	BCrypt with cost factor 11 — resistant to brute force and rainbow tables
Access Token	JWT signed with HMAC-SHA256, expires in 60 minutes, ClockSkew = Zero
Refresh Token	128-bit cryptographically random, expires in 7 days, rotated on use
Token Validation	Validated once at API Gateway — downstream services trust the gateway
Role-Based Access	Claims-based: User role for standard APIs, Admin role for management APIs
Audit Logging	Every auth event (login, failure, KYC) written to AuditLogs table

8.2 API Security

Control	Implementation
Rate Limiting	Per-route limits via Ocelot: 5/min register, 10/min login, 5/sec payments
Input Validation	FluentValidation on all request DTOs — multiple errors returned simultaneously
CORS	Restricted to explicit origin (localhost:4200) — AllowAnyOrigin disabled
Error Messages	Known errors expose message; unknown errors return generic 500 message
ID Security	All IDs are GUIDs — non-sequential, non-enumerable

8.3 Data Security

Concern	Approach
Passwords	Never stored in plain text — BCrypt hash only

Card Data	Tokenization concept — raw card numbers never stored, only tokens
Sensitive Fields	PasswordHash excluded from all DTOs — cannot leak via API response
Audit Trail	Immutable AuditLog and LedgerEntry tables — no UPDATE or DELETE permitted
SQL Injection	Entity Framework Core parameterizes all queries — no raw SQL

9. Key Design Patterns

9.1 Patterns Applied

Pattern	Applied In	Purpose
Repository Pattern	All services	Isolate data access, enable unit testing with mocks
Dependency Injection	All services	Loose coupling, testability, lifetime management
DTO Pattern	All controllers	Decouple API contract from database schema, hide sensitive fields
Middleware Pipeline	All services	Centralized exception handling and logging
BackgroundService	Wallet, Rewards, Notify	Long-running RabbitMQ consumers on dedicated threads
Saga (Choreography)	Transaction service	Distributed transaction rollback without central coordinator
Idempotency Pattern	Transaction service	Prevent duplicate payments on client retry
Guard Clause	Rewards, Wallet	Early return on invalid state, flat code structure
Circuit Breaker	API Gateway	Fast failure after repeated downstream errors
Dead Letter Queue	All consumers	Preserve failed messages for inspection and replay
Outbox Pattern	Event publishers	Publish events after DB commit, failure does not block operation
Tolerant Reader	Rewards consumer	Parse event fields leniently, ignore unknown transaction types

10. Scalability Considerations

10.1 Horizontal Scaling

Each microservice is stateless and can be scaled horizontally by running multiple instances. JWT validation is stateless (no session state). Database connections use connection pooling via EF Core. RabbitMQ consumers in multiple instances compete for messages — only one instance processes each message (competing consumers pattern).

10.2 Scaling Bottlenecks

Component	Scaling Strategy
Auth.API	Stateless JWT — scale freely behind load balancer
Wallet.API	Row-level locking needed for balance updates at scale
Transaction.API	Idempotency table growth — needs TTL cleanup job in production
Rewards.API	Points calculation is CPU-light — scales easily
RabbitMQ	Clustered RabbitMQ with quorum queues for production HA
SQL Server	Read replicas for reporting, connection pooling for OLTP

10.3 Performance Optimisations Applied

- Database indexes on all frequently queried columns (Email, UserId, CreatedAt, IdempotencyKey)
- Async/await throughout — no thread blocking on I/O operations
- EF Core EnableRetryOnFailure — transient SQL errors handled automatically
- RabbitMQ Singleton publisher — one TCP connection shared across all requests
- Separate read queries optimised per use case (EmailExistsAsync vs GetByEmailAsync)
- Event-driven processing — slow operations (notifications, rewards) never block payment response

11. Deployment Architecture

11.1 Current (Development)

Component	Deployment
Services	8 separate processes on Windows, started via start.ps1
Database	SQL Server Express (local), 7 databases on single instance
Message Broker	RabbitMQ Windows service on localhost:5672
Frontend	Angular dev server on localhost:4200 (ng serve)
Gateway	Ocelot process on localhost:5000, started last

11.2 Production Recommendation

Component	Recommended Deployment
Services	Docker containers orchestrated by Kubernetes or Azure AKS
Database	Azure SQL or AWS RDS — managed, backed up, geo-replicated
Message Broker	CloudAMQP or Azure Service Bus — managed RabbitMQ cluster
Frontend	Azure Static Web Apps or AWS CloudFront + S3
Gateway	Behind Azure Application Gateway or AWS ALB with WAF
Secrets	Azure Key Vault or AWS Secrets Manager — no secrets in appsettings

11.3 CI/CD Pipeline (Recommended)

- GitHub Actions triggered on pull request — build + unit tests
- Docker image built per service — tagged with commit SHA
- Integration tests run against Docker Compose stack
- Helm chart deployment to Kubernetes staging environment
- Manual approval gate before production deployment
- Database migrations run as Kubernetes init containers before service start

12. Known Limitations & Roadmap

12.1 Current Limitations

Limitation	Impact	Production Fix
Stock race condition in Catalog	Oversell on concurrent redemptions	SQL UPDATE WHERE StockCount > 0 with row lock
Notification email is simulated	No real emails sent	SendGrid or AWS SES integration
No SMS provider	No real SMS sent	Twilio integration
KYCApprovedEvent missing email	Notification uses placeholder email	Include email in KYCApprovedEvent or Auth lookup
No token blacklisting	Logout does not invalidate JWT	Redis-based JWT blacklist using Jti claim
Idempotency records never purged	Table grows indefinitely	Background cleanup job for expired records
No 2FA	Single factor authentication	TOTP or SMS OTP for high-value operations

12.2 Feature Roadmap

- Docker Compose for one-command local startup
- Kubernetes Helm charts for production deployment
- Real-time balance updates via SignalR WebSocket
- Merchant payment flow with QR code generation
- Card tokenization via Stripe or Razorpay
- PDF transaction statement export
- Admin dashboard Angular UI
- Two-factor authentication (TOTP)
- API versioning (v1, v2)
- OpenTelemetry distributed tracing

13. Glossary

Term	Definition
API Gateway	Single entry point for all client requests — routes, validates JWT, applies rate limits
BCrypt	Password hashing algorithm with built-in salt and configurable cost factor
Circuit Breaker	Pattern that stops forwarding requests to a failing service for a recovery period
Dead Letter Queue (DLQ)	Queue where failed messages are sent after maximum retry attempts
Double-Entry Accounting	Every transaction creates a debit and a credit entry — sum always equals zero
DTO	Data Transfer Object — shapes data for API request/response without exposing entities
Idempotency Key	Client-supplied identifier ensuring the same request is processed only once
Immutable Ledger	Financial records that can only be appended, never updated or deleted
JWT	JSON Web Token — signed, self-contained token for stateless authentication
KYC	Know Your Customer — identity verification process required for financial services
RabbitMQ	Message broker implementing AMQP — delivers messages between services asynchronously
Saga Pattern	Sequence of local transactions with compensating actions on failure
Scoped Lifetime	Dependency injection lifetime — one instance per HTTP request
Singleton Lifetime	Dependency injection lifetime — one instance for entire application lifetime

14. Document Control

Attribute	Value
Document Title	WalletPlatform High Level Design Document
Version	1.0
Status	Final
Platform	.NET 8 | Angular 17 | SQL Server | RabbitMQ | Ocelot
Architecture	Microservices with Event-Driven Communication
Services Count	8 microservices + 1 API Gateway
Databases	7 independent SQL Server databases
Classification	Confidential — Training Project

End of Document